```
NUMBER          RN 0
CNT             RN 1
TMP             RN 2
TMP1            RN 3
COUNT           RN 4
PREV            RN 5
RESULT          RN 6
MASK            RN 7
CNT1            RN 8
TMP2            RN 10
MASK1           RN 11
                AREA MY_CODE,CODE, READONLY
Look_and_Say PROC
                EXPORT Look_and_Say
                PUSH{R4-R8,R10-R11,LR}
                LDR CNT,=0
                LDR CNT1,=1                 ;EDITED: CHANGE CNT1 VALUE FROM 0 TO 1
                LDR COUNT,=0
                LDR PREV,=0
                LDR RESULT,=0
FOR
                LDR TMP,=10
                UDIV TMP1,NUMBER,TMP
                ORR MASK,TMP1,NUMBER    ;EDITED: ADD - TO CHECK IF THE QUOTIENT, THE NUMBER, AND
THE REMAINDER ARE ALL ZERO IN ORDER TO FINISH THE LOOP
                ORR MASK,MASK,TMP2          ;EDITED: ADD - TO CHECK IF THE QUOTIENT, THE NUMBER, AND
THE REMAINDER ARE ALL ZERO IN ORDER TO FINISH THE LOOP
                CMP MASK,#0                 ;EDITED: CHNAGE TMP1 TO MASK
                BEQ FINISH
                MUL TMP,TMP1
                SUB  TMP2,NUMBER,TMP
                MOV NUMBER,TMP1
                CMP CNT,#0
                ITTE  EQ
                MOVEQ PREV,TMP2
```

```
                    ADDEQ COUNT,#1
                    BNE     NOT_ZERO
                    B NEXT
NOT_ZERO
                    CMP PREV,TMP2
                    BEQ     CHECK_PREV
                    LDR MASK,=10            ; EDITED: MOVE
;                   CMP CNT1,#0             ; EDITED: REMOVE
;                   IT          EQ          ; EDITED: REMOVE
;                   LDREQ MASK,=10          ; EDITED: MOVE
;                   MULNE MASK,MASK1        ; EDITED: REMOVE
                    MUL COUNT,MASK
                    ADD COUNT,PREV
                    MUL COUNT,CNT1          ; EDITED: ADD -  (1 * 39 , 100 * 18 , 10000 * 26 , 1000000 * 13)
                    ADD RESULT,COUNT
                    ;ADD CNT1,#2            ; EDITED: REMOVE
                    LDR MASK1,=100          ; EDITED: MOVE  AND SET VALUE TO 100
                    MUL CNT1,MASK1          ; EDITED: ADD - EACH TIME CNT1 MULTIPLY BY 100
                    LDR COUNT,=0
                    MOV PREV,TMP2
CHECK_PREV
                    ADD COUNT,#1
                    B NEXT
NEXT
                    ADD CNT,#1
                    B FOR
FINISH
                    MOV R0,RESULT
                    POP{R4-R8,R10-R11,PC}
                    ENDP
                    END
```

```
#include "Main.h"

extern volatile uint16_t ADC_current;

volatile uint16_t prev_adc = 0;                    // EDITED: ADD – STORE THE PREVIOUS ADC VALUE

extern volatile int change;                        // EDITED: ADD – SET 1  IN IRQ-ADC

int main(){

        SystemInit();

        init_timer_SRI(3,25000,0b011);             // EDITED: ADD - DEBOUNCING

        enable_timer(3);                           // EDITED: ADD - DEBOUNCING

        BUTTON_init();

        LED_init();

        ADC_init();

        ADC_start_conversion();

        while(1){

                __WFI();                // EDITED:  ADD – WAKE UP ONLY WHEN AM INTERRUPT  HAPPENS

                if(change){                        // EDITED:  ADD – SET TO 1 IN IRQ

                        change = 0;                // EDITED:  ADD

                        if (prev_adc != ADC_current){          // EDITED:  ADD

                                LED_Out(0);                    // EDITED:  ADD

                                prev_adc = ADC_current;        // EDITED:  ADD

                                int msb = ADC_current >> 4;    // EDITED:  CHANGE 8 TO 4

                                for(int i=0;i<8;i++){                  // EDITED:  ADD

                                        int led = 11-(4+i);            // EDITED:  ADD

                                        int bit = msb >> (7-i);        // EDITED:  ADD

                                        int val = bit & 1 ;            // EDITED:  ADD

                                        if(val == 1)                   // EDITED:  ADD

                                            LED_On(led);               // EDITED:  ADD

                                }

                        }

                }

        }

}
```

```
uint32_t last_tick0 = 0;
int state0 = 1;
extern uint32_t Look_and_Say(uint32_t digits);
extern volatile uint16_t ADC_current;
int enable = 0;                 // EDITED: ADD - FLAG TO DETECT A BUTTON PRESS AND DISABLE ADC UPDATES
void EINT0_IRQHandler (void)
{
        enable = 1;             // EDITED: ADD - SET TRUE
        LED_Out(0);             // EDITED: ADD
        if(tick<debounce_time && state0==1){
                state0=0;
                last_tick0 = tick;
                int msb = ADC_current >> 4;
                uint32_t result = Look_and_Say(msb);
                uint8_t lsb = (uint8_t)(result & 0xFF);     // EDITED: ADD
                for(int i=0;i<8;i++){                       // EDITED: ADD
                        int led = 11-(4+i);                 // EDITED: ADD
                        int bit = lsb >> (7-i);             // EDITED: ADD
                        int val = bit & 1 ;                 // EDITED: ADD
                        if(val == 1)                        // EDITED: ADD
                                LED_On(led);                // EDITED: ADD
                }
                LPC_SC->EXTINT &= (1 << 0); /* clear pending interrupt      */
                return;
        }
        if ((tick - last_tick0) < debounce_time) {
    LPC_SC->EXTINT &= (1 << 0); /* clear pending interrupt      */
    return;
    }
    last_tick0 = tick;
    state0=1;
```

```
        int msb = ADC_current >> 4;
        uint32_t result = Look_and_Say(msb);
        uint8_t lsb = (uint8_t)(result & 0xFF);      // EDITED: ADD
        for(int i=0;i<8;i++){                        // EDITED: ADD
                int led = 11-(4+i);                  // EDITED: ADD
                int bit = lsb >> (7-i);              // EDITED: ADD
                int val = bit & 1 ;                  // EDITED: ADD
                if(val == 1)                         // EDITED: ADD
                        LED_On(led);                 // EDITED: ADD
        }
        LPC_SC->EXTINT = (1 << 0);    /* clear pending interrupt       */
}
void EINT1_IRQHandler (void)
{
        enable = 0;              // EDITED: ADD - ENABLE BUTTON AND ADC UPDATES
        LPC_SC->EXTINT = (1 << 1);    /* clear pending interrupt       */
}
unsigned short ADC_current;
extern int enable;                   // EDITED: ADD
int change = 0;                      // EDITED: ADD
void ADC_IRQHandler(void) {
        if(enable == 0){             // EDITED: ADD
                ADC_current = ((LPC_ADC->ADGDR>>4) & 0xFFF);
                change = 1;                  // EDITED: ADD
        }
}
```

First, I divide the number by 10 repeatedly to extract the rightmost digit (remainder) and update the number to the quotient. I keep two variables:

- PREV = the previous digit
- COUNT = how many times that digit repeats consecutively

When the remainder becomes different from PREV, it means one group finished, so I create a two-digit group:

- group = (COUNT * 10) + PREV (example: 3 times 9 → 39)

Because I am reading digits from right to left, I store the output in RESULT using a multiplier CNT1:

- First group × 1
- Second group × 100
- Third group × 10000
- Fourth group × 1000000
  (so each time CNT1 = CNT1 * 100, because every group is 2 digits)

Number: 3668999

Group 1 (digit 9):

- 3668999 / 10 = 366899 remainder 9
- 366899 / 10 = 36689 remainder 9
- 36689 / 10 = 3668 remainder 9
  COUNT = 3, digit = 9 → group = 39
  RESULT = 39 × 1 = 39
  CNT1 becomes 100

Group 2 (digit 8):

- 3668 / 10 = 366 remainder 8
  COUNT = 1, digit = 8 → group = 18
  RESULT = 39 + (18 × 100) = 1839
  CNT1 becomes 10000

Group 3 (digit 6):

- 366 / 10 = 36 remainder 6
- 36 / 10 = 3 remainder 6
  COUNT = 2, digit = 6 → group = 26
  RESULT = 1839 + (26 × 10000) = 261839
  CNT1 becomes 1000000

Group 4 (digit 3):

- 3 / 10 = 0 remainder 3
  COUNT = 1, digit = 3 → group = 13
  RESULT = 261839 + (13 × 1000000) = 13261839

1)

- The ADC produces a 12-bit digital value stored in ADC_current.
- In the main loop, the CPU stays in low-power mode using __WFI() and wakes up only when an interrupt occurs.
- When an ADC conversion completes, ADC_IRQHandler() reads the conversion result from LPC_ADC->ADGDR and sets a flag (change = 1) to notify the main loop.
- The main loop checks change and updates LEDs only when a new ADC value is available.
- The 8 most significant bits of the 12-bit ADC result are computed by shifting right 4 positions.
- These 8 bits are displayed on LEDs with the required mapping:
    - LED4 shows the MSB of the 8-bit value
    - LED11 shows the LSB of the 8-bit value
      This is done by checking each bit and turning on the corresponding LED.

2)

- When the user presses INT0, the ISR EINT0_IRQHandler() is executed.
- The routine takes the last ADC-based displayed value (again computed as ADC_current >> 4) and passes it to the assembly subroutine:

  result = Look_and_Say(msb8)
- The output is a 32-bit value; the system extracts the 8 least significant bits using:

  lsb = result & 0xFF
- The 8-bit lsb8 is then displayed on LED4..LED11.
- enable is used to disable ADC updates during the INT0 so the computed result stays visible until the user re-enables ADC updates (INT1).